



Programação de Aplicativos

Professor: Euclides Paim
euclidespaim@gmail.com



Funções em Python

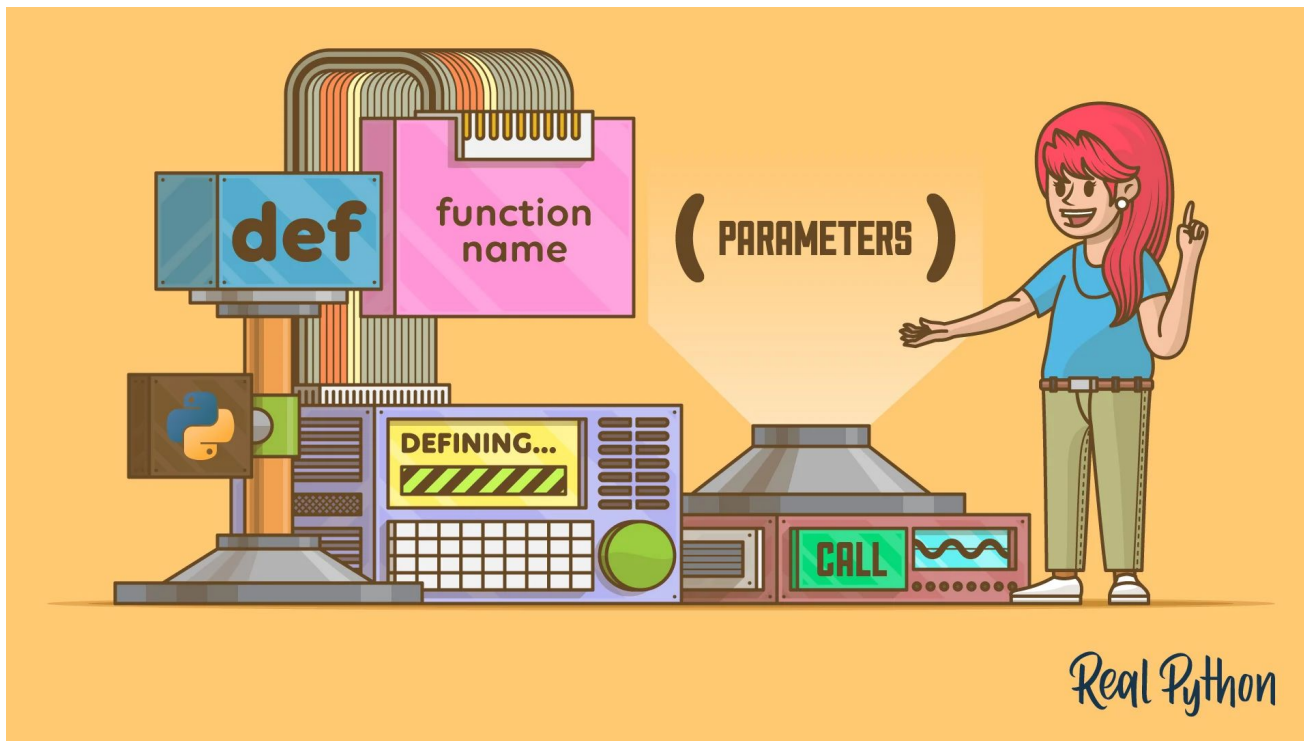
Functions

Professor: Euclides Paim
euclidespaim@gmail.com



Programação de Aplicativos

Funções em Python





Programação de Aplicativos

Laços de Repetição

Sumário

- Definição
- *Functions* sintaxe
- Criando uma função
- Chamando uma função
- Argumentos





Programação de Aplicativos

Functions em Python

Functions em Python

Uma função é um bloco de código que só é executado quando é chamado.

- Você pode passar dados, conhecidos como **parâmetros**, para uma função.
- Uma função pode **retornar** dados como resultado.



Programação de Aplicativos

Functions em Python

Criando uma função

Em Python declaramos uma função usando a palavra reservada **def**,

Sintaxe:

```
def my_function():  
    print("Olá de uma função!!")
```



Programação de Aplicativos

Functions em Python

Chamando uma função

Para chamar uma função, use o nome da função seguido de parênteses:

Sintaxe:

```
def my_function():  
    print("Olá de uma função!!")
```

```
my_function()
```



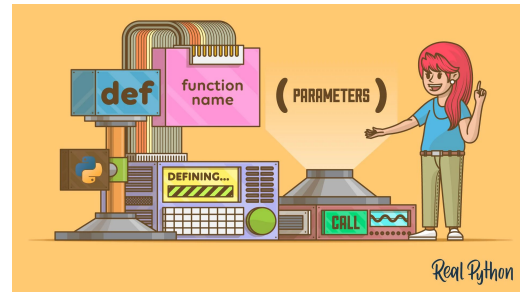
Programação de Aplicativos

Functions em Python

Argumentos

Informações podem ser passadas para funções na forma de **argumentos**.

Os argumentos são especificados após o nome da função, dentro dos parênteses. Podemos adicionar quantos argumentos forem precisos, basta separá-los com vírgula.





Programação de Aplicativos

Functions em Python

Argumentos

O exemplo a seguir possui uma função com um argumento (**fname**). Quando a função é chamada, passamos um primeiro nome, que é usado dentro da função para imprimir o nome completo:

```
def my_function(fname):  
    print(fname + " da Silva")  
  
my_function("Amarildo")  
my_function("Tobias")  
my_function("Jairo")
```

Nota: Argumentos frequentemente são abreviados como **args** na documentação do Python.



Programação de Aplicativos

Functions em Python

Parâmetros ou Argumentos?

Os termos **parâmetro** e **argumento** podem ser usados para se referir à mesma coisa: informações que são passadas para uma função.

Do ponto de vista de uma função:

- Um **parâmetro** é a variável listada entre parênteses na definição da função.
- Um **argumento** é o valor que é enviado para a função quando ela é chamada.



Programação de Aplicativos

Functions em Python

Número de Argumentos

Por padrão, uma função deve ser chamada com a quantidade correta de argumentos. Isso significa que, se a sua função espera 2 argumentos, você deve chamá-la com 2 argumentos — nem mais, nem menos.

```
def my_function(fname, lname):  
    print(fname + " " + lname)  
  
my_function("Tobias", "da Silva")
```

Nota: Se tentarmos chamar a função com 1 ou 3 argumentos, ocorrerá um erro:



Programação de Aplicativos

Functions em Python

Argumentos Arbitrários, *args*

Se não soubermos quantos argumentos serão passados para a nossa função, podemos usar um *** antes do nome do parâmetro na definição da função.

Dessa forma, a função receberá uma tupla de argumentos e poderá acessar os itens conforme necessário:

```
def my_function(*kids):  
    print("A criança mais jovem é " + kids[2])  
  
my_function("Amarildo", "Tobias", "Jairo")
```

Nota: Argumentos Arbitrários frequentemente são abreviados como **args* na documentação do Python.



Programação de Aplicativos

Functions em Python

Passando uma Lista como Argumento

Podemos enviar **qualquer** tipo de dado como argumento para uma função (string, número, lista, dicionário etc.), e ele será tratado como o mesmo tipo de dado dentro da função.

Por exemplo, se enviarmos uma **lista** como argumento, ela continuará sendo uma lista quando chegar à função:

```
def my_function(food):  
    for x in food:  
        print(x)  
  
fruits = ["apple", "banana", "cherry"]  
  
my_function(fruits)
```



Programação de Aplicativos

Functions em Python

Valores de Retorno

Para fazer uma função retornar um valor, use a instrução `return`:

```
def my_function(x):  
    return 5 * x  
  
print(my_function(3))  
print(my_function(5))
```

- A instrução `return` encerra a execução da função e “devolve” um valor ao local onde a função foi chamada.
- Se não usarmos `return`, a função retorna automaticamente `None`.



That's all Folks!



Referências

Referências Básicas

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. ***Algoritmos: teoria e prática***. 3. ed. Rio de Janeiro: Elsevier, 2013.

SEBESTA, Robert W. ***Conceitos de linguagens de programação***. 10. ed. São Paulo: Pearson, 2018.

MANZANO, José Augusto N. G.; OLIVEIRA, Jayr Figueiredo de. ***Algoritmos: lógica para desenvolvimento de programação de computadores***. 27. ed. São Paulo: Érica, 2016.

DOWNEY, Allen B. ***Pense em Python: como pensar como um cientista da computação***. Tradução de Cássio de Souza Costa. 2. ed. São Paulo: Novatec, 2016.

Referências Complementares

IEPSEN, Edécio Fernando. ***Lógica de programação e algoritmos com JavaScript***. 2. ed. São Paulo: Novatec, 2022.

Referências na Internet

<https://www.w3schools.com/python/>